

Module 2: Synchronization

2.1 Global State and Clock Synchronization

A major challenge in distributed computing is the lack of a global clock or shared memory. Every machine's internal clock drifts at a slightly different rate, so even if you synchronize all clocks perfectly at noon, by evening they will have drifted apart by milliseconds or more. This makes it incredibly difficult to determine the exact state of the entire system at any single microsecond, or even to answer a simple question like "which of these two events happened first?"

Physical clock synchronization protocols like **NTP (Network Time Protocol)** reduce drift but can never eliminate it completely. So instead of trying to perfectly synchronize real time, distributed systems often use **logical clocks**, which only care about relative ordering, not exact time.

Lamport Logical Clocks

Lamport's algorithm assigns every event a counter value using two simple rules: (1) increment your local counter before every event, and (2) when you send a message, attach your counter value; when you receive a message, set your counter to $\max(\text{local}, \text{received}) + 1$. This guarantees that if event A causally happened before event B, then A's timestamp is smaller than B's.

■ Lamport Clock in Action

Process P1 sends a message at its local counter 3. Process P2 was at counter 2 when it receives the message. P2 updates its counter to $\max(2, 3) + 1 = 4$. This tells us P2's next event is guaranteed to be logically "after" P1's send — even though P2's raw clock was technically behind.

Vector Clocks extend this idea by giving each process its own counter for every other process in the system (a whole vector of numbers, not just one). This lets the system detect not just "happened before" relationships, but also when two events are *concurrent* (neither one caused the other) — something a single Lamport counter cannot distinguish.

2.2 Election Algorithms

Many distributed systems designate a single node as a "coordinator" to manage tasks such as sequencing transactions or acting as a lock manager. If that node crashes, the system must autonomously detect the failure and elect a new leader — without any human stepping in.

The Bully Algorithm

Every process has a unique ID, and the process with the **highest ID always wins**. When a process notices the coordinator is unresponsive, it "bullies" its way to the top:

- It sends an ELECTION message to all processes with a higher ID.
- If nobody with a higher ID responds, it declares itself the new coordinator and announces this to everyone.
- If a higher-ID process does respond, that process takes over the election, and the original process backs off.

The Ring Algorithm

Nodes are arranged in a logical circle (not necessarily the physical network topology). When a node detects a coordinator failure, it starts passing an election message around the ring, and every node that receives it adds its own ID to the message before forwarding it onward. Once the message travels all the way around back to its originator, that node picks the highest ID from the collected list and announces the winner around the ring a second time.

Feature	Bully Algorithm	Ring Algorithm
Topology assumption	Any process can message any other	Logical ring, one neighbor at a time
Message complexity	Can be high ($O(n^2)$ worst case)	Predictable, $O(n)$ per election
Winner	Highest ID	Highest ID collected around the ring
Speed to converge	Can be fast if IDs are known widely	Bounded by ring traversal time

2.3 Distributed Mutual Exclusion

When multiple nodes need to write to a shared resource, mutual exclusion ensures that only one node gets access at a time to prevent data corruption. Three common strategies are:

- **Centralized:** One coordinator hands out permission on request — simple, but the coordinator is a single point of failure.
- **Token Ring:** A special "token" message circulates around a logical ring; only the holder of the token may access the resource.
- **Ricart-Agrawala (permission-based):** A node wanting access asks every other node for permission and waits for all replies before proceeding — no central coordinator needed, but more messages are exchanged.

■ The Shared Database Analogy

Imagine two administrators simultaneously trying to update the exact same record in a Staff Profile Portal. If the system lacks mutual exclusion, one admin's update might overwrite the other's without warning. Using distributed mutual exclusion (like a Token Ring), only the admin holding the digital "token" is granted permission to write to the database. Once done, they pass the token along.

2.4 Distributed Transactions

Transactions that span across multiple databases or nodes must retain strict **ACID** properties:

Letter	Property	Meaning
A	Atomicity	The transaction either fully completes, or has no effect at all — no partial updates.
C	Consistency	The transaction moves the system from one valid state to another valid state.
I	Isolation	Concurrent transactions do not interfere with each other's intermediate results.
D	Durability	Once committed, the change survives even a crash immediately afterward.

To enforce isolation, systems commonly use **two-phase locking (2PL)**: a transaction acquires all the locks it needs (growing phase) before it starts releasing any of them (shrinking phase). This ensures partial transaction failures do not leave the database in a corrupted, half-updated state.

■ Key Takeaway

Logical clocks give us ordering without needing perfectly synchronized time. Election algorithms let a system self-heal after losing its coordinator. Mutual exclusion and ACID transactions both exist to prevent one node's work from corrupting another's.

Glossary of Key Terms

Term	Meaning
Logical clock	A counter used to order events without relying on real time.
Coordinator	A designated process that manages a task such as locking or sequencing.
Token	A special message that grants its holder exclusive access to a resource.
Two-phase locking	A concurrency control technique with a growing and shrinking lock phase.
ACID	Atomicity, Consistency, Isolation, Durability — the four transaction guarantees.

What You Should Know

- Why physical time cannot be perfectly synchronized across a network, and how Lamport and vector clocks solve ordering instead.
- How the Bully and Ring algorithms resolve coordinator failures, and their trade-offs.
- The three main strategies for mutual exclusion: centralized, token ring, and permission-based.
- The role of ACID properties in distributed multi-node transactions, and how two-phase locking enforces isolation.